Learn Python

Student Edition

5 two-hour sessions + capstone · runnable examples · trap cheat sheets · self-check quizzes

An accelerated, self-study Python course
for a researcher with no prior coding experience.

Offline companion to the interactive website.
Generated 2026-06-30 · Instructional content original

Learn Python — Student Syllabus

Welcome. This is a fast, ~12-hour path (five two-hour sessions, **each covering two topics**) from "never coded" to "I can write a real Python program to wrangle my research data." It's re-ordered for you and front-loaded with the language quirks that trip people up.

How each 2-hour session runs: two halves around a short break. Each half is Concept (≈22 min) → I code, you watch (≈12 min) → **you code (≈25 min)**, then a combined recap + quiz at the end. The practice is the biggest part on purpose — you learn by typing. You will *type every example yourself*.

What you need: Python 3.11+, VS Code (or any editor), and this folder. Open the matching `slides/`, `examples/`, and `cheatsheets/` file for each session. (A full **PDF** of the course is downloadable from the website, and every session is also a **Jupyter notebook** you can run in the browser — no install.)

The 5 sessions (2 hours each, two topics per session)

#	Title	You'll be able to…	Files
1	**Running Python, Types & the Type Traps**	Run code, use `int/float/str/bool/None` & f-strings; tell `==` from `is`, predict `True==1` / `0.1+0.2` / `5=="5"`, check types right	`slides/session-01` · `examples/session-01`
2	**Control Flow & Data Structures**	`if/elif/else`, chained comparisons, `for` / `while`, `enumerate` / `zip`; `list/tuple/dict/set`, comprehensions, sorting, aliasing	`slides/session-02` · `examples/session-02`
3	**Functions, Scope & Recursion**	Write reusable functions, `*args/**kwargs`, type hints, dodge the mutable-default bug; write functions that call themselves, trace the call stack, recurse over nested data	`slides/session-03` · `examples/session-03`
4	**Exceptions, Files & Research Data**	Validate messy input with `try/except` + a first test; read/write CSV survey data, use `statistics` / `datetime`, `pip install`, pandas teaser	`slides/session-04` · `examples/session-04`
5	**Regular Expressions, Modules & OOP**	Validate, extract, and clean real text with regex; import modules, build a small class with `@property`, use generators/ `map` / `filter` /walrus	`slides/session-05` · `examples/session-05`
6	**Capstone (optional)**	Build a Gradebook & Survey Analyzer end-to-end	`assessments/capstone-project`

Session 1's **type traps** (Part B) are the most load-bearing material in the course — if you deeply master one half, make it that one.

Your standing toolkit (open these any time)

- **`cheatsheets/traps-and-gotchas.md`** — every quirk, with the wrong vs right way. *Keep this open.*
- **`cheatsheets/quick-reference.md`** — syntax you'll forget (slicing, f-strings, comprehensions).
- **`cheatsheets/glossary.md`** — plain-language definitions of every term.

How to study (specific to this material)

1. **Type, don't read.** Re-type every example in `examples/`; change one thing and predict the result *before* you run it.
2. **Predict-then-run on traps.** For Session 1's Part B especially, guess the output, then run. The surprise *is* the lesson.
3. **Do both halves' practice.** Each session is sized for a fast learner — aim to finish Part A and Part B, not just the first few tasks.
4. **Keep a "bugs I hit" log.** When you get a traceback, write the last line + your fix. You'll build your own personalized cheat sheet.
5. **Connect to what you know.** Every topic maps to research methods/stats (see `connection-map.md`). Lean on those bridges.

What "done" looks like

You finish the capstone (S6) or, at minimum, complete every session's practice and score the per-session quizzes in `assessments/`. The real test: you can open a messy CSV of your own data and write a short script that summarizes it without copying from anyone.

Scope (so you're not surprised)

This makes you a confident *programmer who handles research data*. It is **not** a full data-science course — pandas, plotting, and statistical modeling get a taste, not a deep dive. Those are the natural next step once these fundamentals are solid.

Session 1

Running Python, Types & the Type Traps

Learn Python — a two-hour session, in two halves.

Part A: Running Python, Variables & Types · **Part B:** The Dynamic-Typing Traps

Part A

Running Python, Variables & Types

Why Python (for you)

- Free, readable, the lingua franca of research computing.
- Glue between your data, your stats, and your writing.
- Today: from zero to "I ran a program."

> Mantra for the whole course: **readability wins**.

Two ways to run Python

1. **REPL** — type `python3`, get `>>>`, run one line at a time. Great for experiments.
2. **Script** — save `hello.py`, run `python3 hello.py`. Great for real work.

```
# hello.py
print("Hello, researcher")
```

Variables = labels on objects

```
n = 30          # an int
mean = 3.7      # a float
name = "Ada"    # a str
passed = True   # a bool
missing = None  # "no value"
```

`=` is an **action** ("stick this label on that object"), **not** a math equation. So `n = n + 1` is fine.

 In stats you name quantities (`n`, `α`). Same idea — but the name can be re-pointed.

The 5 core types

Type	Example	Think
`int`	`30`	counts
`float`	`3.7`	measurements
`str`	`"Ada"`	text
`bool`	`True` / `False`	flags
`NoneType`	`None`	missing

`type(x)` tells you which.

Input & output

```
name = input("Your name: ")        # ⚠ ALWAYS a str
age  = int(input("Your age: "))    # convert right away
print("Hi", name, "— age", age)
```

The #1 week-one trap: `input()` gives you text. `"5" + "3"` is `"53"`, not `8`.

f-strings (use these)

```
score = 87.456
print(f"{name} scored {score:.1f}")   # one decimal
print(f"{1234567:,}")                  # 1,234,567
print(f"{0.873:.1%}")                  # 87.3%
```

Cleaner than `+` concatenation and no type errors.

Type conversion (casting)

```
int("42")     # 42
float("3.14") # 3.14
str(42)       # "42"
int(3.9)      # 3   (truncates, doesn't round!)
round(3.9)    # 4
```

`int(input(...))` = read text, convert to number, in one step.

Reading a traceback (don't panic)

```
Traceback (most recent call last):
  File "x.py", line 3, in <module>
    age = int(input("Age: "))
ValueError: invalid literal for int() with base 10: 'thirty'
```

Read the LAST line first. It names the problem: `ValueError`, and the bad value `'thirty'`.

Live demo & your turn

- Live: `greet.py`, then a "years to graduation" calculator. We'll break it once on purpose.
- You: `examples/session-01/practice.md` — build a GPA-or-BMI style script.

Traps recap

- `input()` → **always a string**; convert with `int()` / `float()`.
- `print(a, b)` (comma → spaces) vs `print(a + b)` (must be same type).
- `int(3.9)` truncates to `3`; use `round()` to round.

Summary

You can run code, name values, convert types, format output, and read an error. **Next:** *Part B of this session* — the dynamic-typing traps.

Practice — Part A

Type each solution yourself. Predict output before running. Solutions at the bottom.

Task 1 — Warm-up REPL

In the REPL (`python3`), check the type of: `42`, `42.0`, `"42"`, `True`, `None`, `7/2`, `7//2`. *What surprises you?* (Hint: `7/2`.)

Task 2 — GPA reporter (`gpa.py`)

Ask for a name and a GPA (a decimal). Print: `"<name>'s GPA is <gpa to 2 decimals>, which is <87.5%> of a 4.0 scale."`

Task 3 — Survey age bucket (`age.py`)

Ask for an age (integer). Print the age, and the age in "months lived" (age × 12). Then deliberately type `twenty` instead of a number and **read the traceback's last line.**

Task 4 — Stretch: the string trap

Without converting, what does `input("a: ") + input("b: ")` print if you type `2` then `3`? Now fix it so it prints `5`.

Bonus — Pythonic idiom drill

Cover the `# ->` answers, predict each line, then run.

```
a, b = 1, 2
a, b = b, a;          print(a, b)        # -> 2 1   (swap, no temp variable)
print(f"{a + b = }")                     # -> a + b = 3   (self-documenting f-string)
print("CS50".lower(), "  hi ".strip())   # -> cs50 hi
print("@" in "ana@uni.edu", len("data")) # -> True 4   (membership + length)
```

Solutions

```
# Task 2 — gpa.py
name = input("Name: ")
gpa = float(input("GPA: "))
print(f"{name}'s GPA is {gpa:.2f}, which is {gpa/4:.1%} of a 4.0 scale.")

# Task 3 — age.py
age = int(input("Age: "))
print(f"You are {age} years old, about {age*12} months.")
# Typing "twenty" -> ValueError: invalid literal for int() with base 10: 'twenty'

# Task 4
# "2" + "3" -> "23"  (string concatenation)
a = int(input("a: ")); b = int(input("b: "))
print(a + b)          # 5
```

Part B

The Dynamic-Typing Traps

A name is just a label

Python won't lock a name to a type — it can point at an `int` now and a `str` later. That freedom is powerful, but it creates **traps** where the result contradicts your intuition.

> This half is hands-on. Every rule below has a one-line, runnable trap in **Traps — predict, then run** at the bottom of the page. Predict first, then reveal.

`==` value, `is` identity

- `==` asks "same value?" — almost always what you want.
- `is` asks "same object?" — use it only for `None` (and `True` / `False`).
- `b = a` does **not** copy: both names point at one list, so mutating `a` shows through `b`. Copy with `list(a)` or `a[:]`.

□ Same GPA = `==`. Same student = `is`.

Booleans are integers

- `bool` is a *subtype* of `int`, so `True` behaves as `1` and `False` as `0`.
- That means `5 + True` works, and `sum(flags)` counts how many are `True`.
- Identity still differs, and `isinstance(True, int)` is true while `type(True) is int` is not.

□ Dummy coding (1/0) baked right into the language.

Numbers: int, float, division

- `/` **always** returns a float (`4 / 2` is `2.0`); `//` floors **toward −∞**.
- `3 == 3.0` compares by value, but decimals are stored in **binary**, so `0.1 + 0.2` is not exactly `0.3`.
- Never test two floats with `==` — use `math.isclose(a, b)`. (You already never compare two measured scores for exact equality; same instinct.)

Comparing across types

- `==` / `!=` across types returns `False` and never crashes.
- `<` / `>` across incompatible types raises **TypeError** — the computer can ask "same?" but can't *rank* text against numbers.
- A list and a tuple with the same contents are **never equal**; sequences compare element by element.

Truthiness

- Falsy: `0 0.0 "" [] {} set() None`. Truthy: everything else — including `"0"`, `"False"`, and `[0]`.
- Idiom: `if scores:` rather than `if len(scores) > 0:`.
- But convert user text first — `"0"` is truthy.

Now spring the traps

Open **Traps — predict, then run** below: ~18 one-line traps. For each one, **say your prediction out loud, then run it** to reveal the real result and the reason behind it.

Then in `examples/session-01/practice.md`: write `clean_score(value)` that accepts `5`, `5.0`, or `"5"` and returns a float, rejecting nonsense — safely.

Next: Control flow & data structures.

Practice — Part B

Predict-the-output gauntlet

For each line, write *why* (one sentence). Predict, then run to check.

```
True + True            # 2     — bools are ints; True is 1
3 == 3.0               # True  — numbers compare by value
0.1 + 0.2 == 0.3       # False — binary float rounding
5 == "5"               # False — different types, no error
5 > "5"                # ▫     — can't ORDER int vs str
[1,2] == (1,2)         # False — list vs tuple are different types
bool("0")              # True  — non-empty string is truthy
x=[1]; y=x; x.append(2); y   # [1,2] — y is an alias of x
```

Build `clean_score()`

Write a function that safely turns a value into a float on a 0–100 scale:

```
def clean_score(value):
    """
    Accept 87, 87.0, or "87" and return 87.0 (a float).
    - Reject anything outside 0..100 with a clear message (return None).
    - Compare floats safely (no exact ==).
    """
```

Test it on: `87`, `87.0`, `"87"`, `"eighty"`, `120`, `True`. *What does `True` do, and why? (Hint: bool is an int...)*

Bonus — Pythonic idiom drill

Cover the `# ->` answers, predict each line, then run.

```
x = int("257"); y = int("257")
print(x == y, x is y)                # -> True False   (equal value, different objects)
print(float("nan") == float("nan"))  # -> False        (NaN equals nothing, not even
itself)
```

Solutions

```
import math

def clean_score(value):
    # Reject bools explicitly — they'd sneak through as ints (True == 1).
    if isinstance(value, bool):
        print(f"Rejected {value!r}: looks like a flag, not a score.")
        return None
    try:
        score = float(value)              # handles int, float, and numeric strings
    except (ValueError, TypeError):
        print(f"Rejected {value!r}: not a number.")
        return None
    if not 0 <= score <= 100:
        print(f"Rejected {value!r}: out of range 0–100.")
        return None
    return score

for v in [87, 87.0, "87", "eighty", 120, True]:
    print(v, "->", clean_score(v))
# 87->87.0, 87.0->87.0, "87"->87.0, "eighty"->None, 120->None, True->None
```

Key lesson: `float(True)` is `1.0`, so without the explicit bool check a flag would pass as a valid score. This is the `bool ⊂ int` trap in a real function.

Traps — predict, then check

Cover the result, predict each one, then check yourself.

1.

```
a = [1, 2]
b = [1, 2]
a is b
```

Most people expect `True — the lists are equal` — the result is `False`. `==` compares VALUES; `is` compares IDENTITY (the object in memory). Two equal-looking lists are still different objects. Use `==` for values and keep `is` for `None`.

2.

```
a = [1, 2]
b = a
a.append(3)
b
```

Most people expect `[1, 2] — we only touched a` — the result is `[1, 2, 3]`. `b = a` binds the SAME list, not a copy, so mutating through `a` shows up through `b`. Copy with `list(a)` or `a[:]`.

3.

```
True == 1
```

Most people expect `False — they are different types` — the result is `True`. `bool` is a subclass of `int`: `True == 1` and `False == 0`.

4.

```
5 + True
```

Most people expect `TypeError — you can't add a bool to an int` — the result is `6`. `True` acts as `1` (and `False` as `0`) in arithmetic, so `5 + True == 6`.

5.

```
flags = [True, False, True, True]
sum(flags)
```

Most people expect `an error, or 0` — the result is `3`. Summing booleans counts the `True`s — a handy way to count how many tests passed.

6.

```
0.1 + 0.2
```

Most people expect `0.3` — the result is `0.30000000000000004`. Floats are stored in binary, where 0.1 and 0.2 have no exact representation, so the tiny errors add up.

7.

```
0.1 + 0.2 == 0.3
```

Most people expect `True` — the result is `False`. Because `0.1 + 0.2` is `0.30000000000000004`. Never test floats with `==`; use `math.isclose(a, b)`.

8.

```
nan = float('nan')
nan == nan
```

Most people expect `True — it is the very same value` — the result is `False`. `NaN` is defined to be equal to nothing, not even itself. Test it with `math.isnan(x)`.

9.

```
7 / 2
```

Most people expect `3` — the result is `3.5`. `/` is always float (true) division. Use `//` for floor division.

10.

```
-7 // 2
```

Most people expect `-3 (chop toward zero)` — the result is `-4`. `//` floors toward negative infinity, not toward zero, so `-7 // 2 == -4`.

11.

```
5 == '5'
```

Most people expect `True — same digit` — the result is `False`. Python does no automatic number/text conversion, so a number and a string are simply unequal (no error).

12.

```
5 > '5'
```

Most people expect `False (or maybe True)` — the result is `TypeError: '>' not supported between instances of 'int' and 'str'`. Ordering a number against text raises TypeError — there is no sensible order. Convert first: `5 > int('5')`.

13.

```
[1, 2] == (1, 2)
```

Most people expect `True — same elements` — the result is `False`. A list never equals a tuple, whatever the contents.

14.

```
type(True) is int
```

Most people expect `True — bools are ints` — the result is `False`. `type()` is exact and `True`'s type is `bool`, not `int`. Use `isinstance` when you want subclass-aware checks.

15.

```
isinstance(True, int)
```

Most people expect `False — it's a bool, not an int` — the result is `True`. `isinstance` respects subclassing, and `bool` IS a kind of `int`.

16.

```
bool('0')
```

Most people expect `False — it's zero` — the result is `True`. Any NON-EMPTY string is truthy, including '0' and 'False'. Only the empty string is falsy.

17.

```
bool([])
```

Most people expect `True — the list exists` — the result is `False`. Empty containers (`[]`, `{}`, `''`, `0`, `None`) are all falsy.

18.

```
m = int('257')
p = int('257')
m is p
```

Most people expect `True — same number` — the result is `False`. CPython pre-caches small ints (-5..256), so `is` accidentally looks True there; 257 built at runtime are separate objects. Compare values with `==`, never `is`.

Session 2

Control Flow & Data Structures

Learn Python — a two-hour session, in two halves.

Part A: Control Flow: Conditionals & Loops · **Part B:** Data Structures

Part A

Control Flow: Conditionals & Loops

Part 1 — Conditionals

```
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
else:
    grade = "F"
```

Indentation defines the block. Only the **first** true branch runs.

Comparison + chained comparisons

`==` `!=` `<` `<=` `>` `>=`

```
0 <= score <= 100      # chained — reads like math
```

□ No need for `score >= 0 and score <= 100` — chain it.

Logical operators (and the gotcha)

```
passed and submitted     # both
late or excused          # either
not flagged              # negate
```

Short-circuit: `a and b` skips `b` if `a` is falsy. And `and` / `or` return an **operand, not a bool**:

```
5 and 0        # ?  ← predict, then reveal in Traps below
"" or "N/A"    # the default-value idiom
```

So write `if x:` — never `if x == True`.

Part 2 — Loops

```
for s in scores:          # each element directly
    print(s)

for i in range(5):        # 0,1,2,3,4
    print(i)

while not done:           # repeat until a condition flips
    ...
```

⚠ `range(1, 5)` → `1,2,3,4` — **stop is excluded** (off-by-one!).

---

break / continue

```
for x in data:
    if x is None:
        continue          # skip this one
    if x == "STOP":
        break             # leave the loop entirely
    process(x)
```

Stop juggling indices: enumerate & zip

```
for i, name in enumerate(names):       # index + value
    print(i, name)

for name, score in zip(names, scores): # two lists together
    print(name, score)
```

□ If you write `range(len(x))`, stop — use `enumerate` / `zip`.

The validation loop (you'll reuse this everywhere)

```
while True:
    raw = input("Score 0–100: ")
    if raw.isdigit() and 0 <= int(raw) <= 100:
        score = int(raw)
        break
    print("Try again.")
```

Your turn

`examples/session-02/practice.md`:

1. Grade-band classifier — test the boundaries (89.999 / 90 / 90.001).
2. Average a roster and label each student PASS/FAIL with `zip`.
3. A robust "ask until valid" loop.

Traps recap

- `if x == True` → just `if x:`; use `x is None` (not `== None`).
- `=` (assign) vs `==` (compare) — classic typo.
- `range(1, 5)` excludes 5; test your boundaries.
- Don't modify a list while looping it; prefer `enumerate` / `zip` over `range(len(...))`.

(More in the cheat sheet: `match` / `case`, the ternary, `for/else`.)

Summary

You can branch and repeat cleanly. **Next:** *Part B of this session* — data structures.

Practice — Part A

Task 1 — Grade-band classifier

Write `letter_grade(score)` returning A/B/C/D/F (90/80/70/60 cutoffs), `"Invalid"` outside 0–100.
Test the boundaries: 90, 89.999, 0, 100, -5, 101.

Task 2 — Boolean logic

1. What is `5 and 0`? `"" or "N/A"`? Why aren't they `True` / `False`?
2. Rewrite `if attended == True:` the Pythonic way.

Task 3 — Average + pass/fail (loops)

Given `names = ["Ana","Ben","Cara","Dev"]` and `scores = [91, 58, 73, 64]`:

1. Compute the mean with a loop and a running total.
2. Use `zip` to print `"<name>: PASS"` (≥60) or `"<name>: FAIL"`.
3. Count the passes with `sum(s >= 60 for s in scores)`.

Task 4 — Validation loop

Write a real `while True:` prompt that keeps asking until the user types an integer 0–100.

Task 5 — Trap check

Why does this skip elements, and what's the fix?

```
xs = [1, 2, 3, 4]
for x in xs:
    if x % 2 == 0:
        xs.remove(x)
```

Bonus — Pythonic idiom drill

Cover the `# ->` answers, predict each line, then run.

```
nums = [80, 92, 45]
print(all(n >= 60 for n in nums))    # -> False   (45 fails)
print(any(n >= 90 for n in nums))    # -> True    (92 passes)
print(92 in nums, 60 in nums)        # -> True False
```

Solutions

```
# 1
def letter_grade(score):
    if not 0 <= score <= 100: return "Invalid"
    for cutoff, g in [(90,"A"),(80,"B"),(70,"C"),(60,"D")]:
        if score >= cutoff: return g
    return "F"
# 90->A, 89.999->B, 0->F, 100->A, -5->Invalid, 101->Invalid

# 2
# 5 and 0 -> 0 ; "" or "N/A" -> "N/A"  (and/or return an operand, not a bool)
result = "pass" if attended else "absent"     # and just `if attended:`

# 3
total = 0
for s in scores: total += s
print(total / len(scores))                    # 71.5
for name, score in zip(names, scores):
    print(f"{name}: {'PASS' if score >= 60 else 'FAIL'}")
print("passes:", sum(s >= 60 for s in scores))   # 3

# 4
while True:
    raw = input("Score 0–100: ")
    if raw.isdigit() and 0 <= int(raw) <= 100:
        print("Got", int(raw)); break
    print("Try again.")

# 5  Removing while iterating shifts indices, so elements get skipped.
xs = [x for x in xs if x % 2 != 0]            # build a new list instead -> [1, 3]
```

Part B

Data Structures

Four containers, four jobs

Type	Syntax	Mutable?	Use for
`list`	`[1, 2, 3]`	yes	ordered, changing collection
`tuple`	`(1, 2)`	no	fixed record / coordinates
`dict`	`{"k": v}`	yes	key → value lookup
`set`	`{1, 2, 3}`	yes	unique items

Lists & slicing

```
xs = [10, 20, 30, 40]
xs[0]      # 10     xs[-1]   # 40 (last)
xs[1:3]    # [20, 30]   (stop excluded)
xs[:2]     # [10, 20]
xs[::-1]   # reversed
xs.append(50); xs.sort()      # mutate in place
```

⚠ `xs.sort()` returns **None** — it sorts in place. Use `sorted(xs)` for a new list.

Dicts = labeled records

```
student = {"name": "Ana", "gpa": 3.9}
student["name"]              # "Ana"
student.get("major", "N/A")  # safe access with default
student["major"] = "Ed"      # add/update
for key, val in student.items(): ...
```

A list of dicts = a dataset □

```
roster = [
    {"name": "Ana", "score": 91},
    {"name": "Ben", "score": 58},
]
```

Each dict = a **row/respondent**; each key = a **variable/column**. This is your tidy dataset until pandas shows up (Session 4).

Sets: unique, fast membership

```
answers = ["yes", "no", "yes", "maybe", "no"]
set(answers)            # {'yes', 'no', 'maybe'}  — dedup
"yes" in set(answers)   # True, very fast
```

Great for "distinct responses" and "have I seen this ID?"

Comprehensions

```
[s["score"] for s in roster]                # list
[s for s in roster if s["score"] >= 60]     # with filter
{s["name"]: s["score"] for s in roster}     # dict
{s["score"] // 10 for s in roster}          # set of score-decades
```

Read as: *expr, for each item, (optionally) if condition.*

Sorting with a key

```
sorted(roster, key=lambda s: s["score"])               # ascending
sorted(roster, key=lambda s: s["score"], reverse=True) # descending
```

`lambda s: s["score"]` = "sort by the score field."

TRAP: aliasing (labels, not boxes)

```
a = [1, 2, 3]
b = a                # SAME list
a.append(4)
b                    # ?  😱  ← predict (see Traps below)

b = a.copy()         # ✅ independent copy
```

`[[0]*3]*3` makes 3 references to ONE row — use `[[0]*3 for _ in range(3)]`.

Your turn

`examples/session-02/practice.md`:

1. Build the roster (list of dicts); sort by score.
2. `{name: score}` dict comprehension.
3. Group students into pass/fail buckets.
4. Demonstrate the aliasing trap and fix it.

Traps recap

- `=` aliases; use `.copy()` / `copy.deepcopy()`.
- `.sort()` returns None (in place); `sorted()` returns new.
- list ≠ tuple even with same contents (Session 2).
- `dict.get(key, default)` avoids `KeyError`.

Summary

You can store, look up, dedup, sort, and reshape data. **Next:** Functions, scope & recursion.

Practice — Part B

Start from:

```
roster = [
    {"name": "Ana", "score": 91}, {"name": "Ben", "score": 58},
    {"name": "Cara", "score": 73}, {"name": "Dev", "score": 64},
]
```

Task 1 — Rank

Print names sorted by score, highest first.

Task 2 — Map (dict comprehension)

Build `{name: score}` in one line.

Task 3 — Group

Build `{"pass": [...names...], "fail": [...names...]}` using a loop.

Task 4 — Dedup

From `["A","B","A","C","B"]`, get the distinct values and how many there are.

Task 5 — Aliasing

Show that `b = roster` then `roster.append({...})` also changes `b`. Then make `b` an independent copy so it doesn't. (Hint: nested dicts → `copy.deepcopy`.)

Bonus — Pythonic idiom drill

Cover the `# ->` answers, predict each line, then run.

```
head, *tail = [10, 20, 30, 40]
print(head, tail)                    # -> 10 [20, 30, 40]   (star-unpacking)
print({"a": 1} | {"b": 2})           # -> {'a': 1, 'b': 2}  (dict union, 3.9+)
print({1, 2, 3} & {2, 3, 4})         # -> {2, 3}            (set intersection)
print(list(zip(*[(1, 2), (3, 4)])))  # -> [(1, 3), (2, 4)]  (transpose)
```

Solutions

```
# 1
print([s["name"] for s in sorted(roster, key=lambda s: s["score"], reverse=True)])
# ['Ana', 'Cara', 'Dev', 'Ben']

# 2
name_to_score = {s["name"]: s["score"] for s in roster}

# 3
groups = {"pass": [], "fail": []}
for s in roster:
    groups["pass" if s["score"] >= 60 else "fail"].append(s["name"])

# 4
vals = ["A","B","A","C","B"]
distinct = set(vals); print(distinct, len(distinct))   # {'A','B','C'} 3

# 5
import copy
b = roster                        # alias
roster.append({"name": "Eve", "score": 80})
# b now also has Eve. To stay independent:
b = copy.deepcopy(roster)         # changes to roster no longer touch b
```

Traps — predict, then check

Cover the result, predict each one, then check yourself.

1.

```
5 and 0
```

Most people expect `True or False` — the result is `0`. `and` / `or` return one of the OPERANDS, not a bool: `and` yields the first falsy value (else the last), `or` the first truthy.

2.

```
list(range(1, 5))
```

Most people expect `[1, 2, 3, 4, 5]` — the result is `[1, 2, 3, 4]`. `range(start, stop)` stops BEFORE `stop`.

3.

```
grid = [[0] * 3] * 3
grid[0][0] = 9
grid
```

Most people expect `[[9, 0, 0], [0, 0, 0], [0, 0, 0]]` — the result is `[[9, 0, 0], [9, 0, 0], [9, 0, 0]]`. `[[0]*3]*3` makes three references to ONE inner row, so editing one edits all. Build with `[[0]*3 for _ in range(3)]`.

4.

```
scores = [55, 92, 78]
all(s >= 60 for s in scores)
```

Most people expect `True` — the result is `False`. `all()` is True only if EVERY item passes; 55 fails, so it's False.

5.

```
d = {'Ana': 91}
d.get('Ben')
```

Most people expect `KeyError` — the result is `None`. `.get()` returns `None` (or a default) for a missing key instead of raising — unlike `d['Ben']`.

6.

```
nums = [1, 2, 3, 4]
nums[::-1]
```

Most people expect `an error, or the same list` — the result is `[4, 3, 2, 1]`. A slice with step -1 returns a reversed COPY — a common Python idiom.

Session 3

Functions, Scope & Recursion

Learn Python — a two-hour session, in two halves.

Part A: Functions, Scope & Reusability · **Part B:** Recursion & Recursive Thinking

Part A

Functions, Scope & Reusability

Defining & calling

```
def class_average(scores):
    """Return the mean of a list of scores."""
    return sum(scores) / len(scores)

class_average([91, 58, 73])     # 74.0
```

□ A function is a formula/coding-scheme: same input → same output.

return vs print

```
def avg(xs): return sum(xs) / len(xs)   # hands value back
def show(xs): print(sum(xs) / len(xs))  # just displays

x = avg([1,2,3])     # x = 2.0
y = show([1,2,3])    # prints 2.0, but y is None!
```

`print` shows; `return` gives the value to the next step.

Parameters: positional, keyword, default

```
def grade(score, scale=100, passing=60):
    ...
grade(85)                 # uses defaults
grade(85, passing=50)     # keyword arg
```

⚠ Defaults must be **immutable** (numbers, strings, `None`) — never `[]` or `{}`.

*args / *kwargs

```
def total(*args):        # any number of positionals -> tuple
    return sum(args)
total(1, 2, 3)           # 6

def tag(**kwargs):       # any number of keywords -> dict
    return kwargs
tag(name="Ana", gpa=3.9) # {'name':'Ana','gpa':3.9}

func(*my_list)           # unpack list into args
func(**my_dict)          # unpack dict into kwargs
```

TRAP: mutable default argument 😱

```
def add_student(name, roster=[]):    # ✖
    roster.append(name)
    return roster

add_student("Ana")    # ?
add_student("Ben")    # ?  ← does the list persist? (Traps below)
```

The default `[]` is created **once**, at definition. Fix on next slide.

The fix: default to None

```
def add_student(name, roster=None):   # ✔
    if roster is None:
        roster = []
    roster.append(name)
    return roster
```

Rule: mutable default? Use `None` and create inside.

Scope (LEGB) & globals

Python looks up names: **L**ocal → **E**nclosing → **G**lobal → **B**uilt-in.

```
count = 0
def bump():
    count = count + 1   # ▫ UnboundLocalError
```

Assigning `count` makes it local. Avoid `global`; **return** a value and reassign instead.

Docstrings & type hints

```
def class_average(scores: list[float]) -> float:
    """Return the arithmetic mean of `scores`."""
    return sum(scores) / len(scores)
```

Type hints document intent. **They are NOT enforced at runtime** (`mypy` checks them).

Your turn

`examples/session-03/practice.md`:

1. A small grade-functions module (with docstrings + hints).
2. Reproduce the mutable-default bug, then fix it.
3. `summary(*scores)` using `*args`.

Traps recap

- Mutable default arg → use `None`.
- `print` ≠ `return` (forgot return → `None`).
- Assigning a global inside a function → `UnboundLocalError`.
- Type hints aren't enforced.

Summary

You can write reusable, documented, reproducible functions. **Next:** *Part B of this session* — recursion.

Practice — Part A

Task 1 — Grade-functions module

Write three functions with docstrings and type hints:

- `class_average(scores: list[float]) -> float`
- `letter_grade(score: float) -> str` (reuse Session 3)
- `pass_rate(scores: list[float], passing: float = 60) -> float` (fraction passing, 0–1)

Use bool-summing for `pass_rate` (recall `sum(s >= passing for s in scores)`).

Task 2 — Reproduce & fix the mutable-default bug

Write `add_note(text, notes=[])` that appends and returns. Call it three times and watch the list grow. Then fix it with the `None` pattern and prove each call starts fresh.

Task 3 — *args summary

Write `summary(*scores)` that returns a dict `{"n":..., "mean":..., "max":..., "min":...}`. Call it both as `summary(91, 58, 73)` and as `summary(*my_list)`.

Bonus — Pythonic idiom drill

Cover the `# ->` answers, predict each line, then run.

```
def f(a, *, b):          # everything after * is keyword-only
    return a, b
print(f(1, b=2))                    # -> (1, 2)
print(f(**{"a": 1, "b": 9}))        # -> (1, 9)   (** unpacks a dict into arguments)
```

Solutions

```
def class_average(scores: list[float]) -> float:
    """Mean of scores."""
    return sum(scores) / len(scores)

def letter_grade(score: float) -> str:
    """A/B/C/D/F by 90/80/70/60 cutoffs."""
    for cutoff, letter in [(90,"A"),(80,"B"),(70,"C"),(60,"D")]:
        if score >= cutoff:
            return letter
    return "F"

def pass_rate(scores: list[float], passing: float = 60) -> float:
    """Fraction of scores >= passing (0..1)."""
    return sum(s >= passing for s in scores) / len(scores)

# Task 2
def add_note(text, notes=None):     # fixed version
    if notes is None:
        notes = []
    notes.append(text)
    return notes

# Task 3
def summary(*scores):
    return {"n": len(scores), "mean": sum(scores)/len(scores),
            "max": max(scores), "min": min(scores)}
print(summary(91, 58, 73))
print(summary(*[91, 58, 73]))
```

Part B

Recursion & Recursive Thinking

The shape of every recursion

```
def countdown(n):
    if n <= 0:          # BASE CASE — when to stop
        print("liftoff!")
        return
    print(n)
    countdown(n - 1)    # RECURSIVE CASE — same problem, smaller input
```

Two parts, always:

- a **base case** that stops, and
- a **recursive case** that moves *toward* the base case.

Trace the call stack

```
factorial(3)
= 3 * factorial(2)
=     3 * (2 * factorial(1))
=         3 * (2 * 1)        # base case returns 1
= 6
```

Each call waits on the one inside it. The calls stack up, then unwind.

 Each pending call is a **stack frame** — that matters in a moment.

Recursion vs iteration

```
def factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n - 1)   # ← must RETURN the call

def factorial_loop(n):
    total = 1
    for k in range(2, n + 1):
        total *= k
    return total
```

Same answer. For flat counting, the **loop** is usually clearer.

Where recursion shines: nested data

```
def deep_sum(obj):
    if isinstance(obj, (int, float)):
        return obj
    if isinstance(obj, dict):
        return sum(deep_sum(v) for v in obj.values())
    if isinstance(obj, (list, tuple)):
        return sum(deep_sum(x) for x in obj)
    return 0

deep_sum([1, [2, [3, 4]], {"a": 6}])   # 16
```

Nested JSON, folder trees, threaded replies — a single loop can't reach all the way down. Recursion can.

The trap: no base case

```
def runaway(n):
    return runaway(n + 1)     # never stops
```

```
RecursionError: maximum recursion depth exceeded
```

Python has **no tail-call optimization** — every call keeps its frame (default limit ≈ 1000). Deep recursion *will* hit the ceiling.

Your turn

`examples/session-03/practice.md`:

1. Recursive `rsum(n)` — name the base case first.
2. `flatten([1, [2, [3, 4]], 5])` → one flat list.
3. `depth(...)` — how deeply is a list nested?

Traps recap

- Every recursion needs a **reachable base case**, or it overflows the stack.
- **Return** the recursive call — forgetting to gives you a silent `None`.
- Recursion isn't free: each call costs a stack frame (no tail-call optimization).
- A plain **loop** is better for flat sequences and for very deep work.

Summary

You can solve problems that are defined in terms of themselves — especially nested data. **Next:** Exceptions, files & research data.

Practice — Part B

Task 1 — Recursive sum

Write `rsum(n)` that adds `1 + 2 + ... + n` **with recursion** (no loop). Name the base case out loud before you write it. Test `rsum(5)` and `rsum(0)`.

Task 2 — Recursion vs iteration

Write `reverse(s)` that reverses a string recursively. Then write the loop version. Which reads more clearly to you? Test `reverse("data")`.

Task 3 — Flatten nested data

Write `flatten(xs)` that turns a list-of-lists (nested to any depth) into one flat list: `flatten([1, [2, [3, 4]], 5])` → `[1, 2, 3, 4, 5]`. This is the move for nested JSON/exports.

Task 4 — How deep does it go?

Write `depth(xs)` returning how deeply a list is nested: `depth([1, [2, [3, [4]]]])` → `4`, `depth([1, 2, 3])` → `1`, `depth(5)` → `0`.

Task 5 — Trap check

1. Why does this raise `RecursionError`, and what's the fix? `python def f(n): return n + f(n - 1)`
2. This returns `None` instead of a number — why? `python def fact(n): if n <= 1: return 1 n * fact(n - 1)`
3. Name one case where a plain loop is the better choice over recursion.

Bonus — Pythonic idiom drill

One decorator makes exponential recursion instant by remembering past calls.

```
import functools

@functools.cache                    # memoize: each n is computed once
def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)
print(fib(35))                      # -> 9227465   (try this WITHOUT @cache... then wait)
```

Solutions

```
# 1
def rsum(n):
    if n == 0:                      # base case
        return 0
    return n + rsum(n - 1)
print(rsum(5), rsum(0))             # 15 0

# 2
def reverse(s):
    if s == "":                     # base case: empty string
        return ""
    return reverse(s[1:]) + s[0]    # all-but-first, reversed, then first
print(reverse("data"))             # "atad"
# loop version: "".join(reversed(s))  — usually clearer for flat strings

# 3
def flatten(xs):
    out = []
    for x in xs:
        if isinstance(x, list):
            out.extend(flatten(x))  # recurse into the sub-list
        else:
            out.append(x)
    return out
print(flatten([1, [2, [3, 4]], 5]))   # [1, 2, 3, 4, 5]

# 4
def depth(xs):
    if not isinstance(xs, list):
        return 0                              # a non-list has no nesting
    return 1 + max((depth(x) for x in xs), default=0)
print(depth([1, [2, [3, [4]]]]), depth([1, 2, 3]), depth(5))   # 4 1 0

# 5
# 1) No reachable base case -> the calls never stop -> stack overflows.
#    Fix: add `if n == 0: return 0` (or n <= 0) at the top.
# 2) The recursive case computes n*fact(n-1) but never RETURNs it,
#    so the function falls off the end and returns None. Add `return`.
# 3) A loop is better when the work is a simple flat sequence, or when the
#    depth could exceed ~1000 (Python has no tail-call optimization, so deep
#    recursion hits RecursionError where a loop would be fine).
```

Traps — predict, then check

Cover the result, predict each one, then check yourself.

1.

```
def add(item, bag=[]):
    bag.append(item)
    return bag
add('apple')
add('banana')
```

Most people expect `['banana'] — a fresh empty bag each call` — the result is `['apple', 'banana']`. A default value is created ONCE, at def time, so the same list persists across calls. Use `bag=None` then `bag = bag or []`.

2.

```
def show(x):
    print(x)
show(42) is None
```

Most people expect `False — it clearly produced 42` — the result is `True`. Printing is not returning. A function with no `return` gives `None`.

3.

```
funcs = [lambda: i for i in range(3)]
[f() for f in funcs]
```

Most people expect `[0, 1, 2]` — the result is `[2, 2, 2]`. Closures capture the VARIABLE `i`, not its value; by call time the loop has left `i = 2`. Fix by binding it: `lambda i=i: i`.

4.

```
def f(n):
    return f(n - 1)
f(3)
```

Most people expect `runs forever, or 0` — the result is `RecursionError: maximum recursion depth exceeded`. Every recursive function needs a base case. Without one Python stops at its recursion limit (~1000 deep) with RecursionError.

Session 4

Exceptions, Files & Research Data

Learn Python — a two-hour session, in two halves.

Part A: Exceptions & Defensive Code · **Part B:** Files, Libraries & Research Data

Part A

Exceptions & Defensive Code

try / except

```
try:
    n = int(value)
except ValueError:
    n = None        # handle the bad case
```

When code might fail at runtime, wrap it. The `except` catches the named error.

Common exception types

Exception	Happens when
`ValueError`	right type, bad value: `int("N/A")`
`TypeError`	wrong type: `5 > "5"`
`KeyError`	missing dict key: `d["nope"]`
`IndexError`	bad list index: `xs[99]`
`ZeroDivisionError`	`x / 0`
`FileNotFoundError`	`open("missing.csv")`

try / except / else / finally

```
try:
    n = int(value)
except ValueError:
    print("not a number")
else:
    print("ok:", n)      # only if NO exception
finally:
    print("always runs")  # cleanup
```

Raise your own

```
def clean_likert(n):
    if not 1 <= n <= 5:
        raise ValueError(f"{n} not in 1–5")
    return n
```

`raise` throws an exception on purpose — caller decides how to handle it.

EAFP vs LBYL

```
# LBYL — "look before you leap"
if value.isdigit():
    n = int(value)

# EAFP — "easier to ask forgiveness" (Pythonic)
try:
    n = int(value)
except ValueError:
    n = None
```

Both valid. EAFP shines when "checking first" is hard or racy.

assert (developer check, not validation)

```
assert len(scores) > 0, "scores must not be empty"
```

For *your* sanity checks while developing. Can be disabled (`python -O`), so **never** use `assert` to validate untrusted input — use `raise`.

A first test with pytest

```
# clean.py
def clean_likert(n):
    if not 1 <= n <= 5:
        raise ValueError("1–5 only")
    return n

# test_clean.py
import pytest
from clean import clean_likert

def test_valid():    assert clean_likert(3) == 3
def test_invalid():
    with pytest.raises(ValueError):
        clean_likert(9)
```

Run: `pytest`

Your turn

`examples/session-04/practice.md`:

1. `safe_int(value)` returning int or None.
2. Clean a dirty survey list, collecting good values + a rejection log.
3. Write one `pytest` test.

Traps recap

- **Never** bare `except:` — name the exception.
- Don't catch too broadly or swallow errors silently.
- `assert` ≠ input validation (use `raise`).
- Catch the *specific* error you expect.

Summary

You can validate messy input and fail loudly when you should. **Next:** *Part B of this session* — files & research data.

Practice — Part A

Task 1 — `safe_int`

Write `safe_int(value)` returning `int(value)` or `None` on failure. Test on `"42"`, `"N/A"`, `""`, `None`, `3.0`.

Task 2 — Clean a survey column

Given `raw = ["5","3","N/A","7","","1","two","4"]`, produce:

- `clean` — list of valid Likert ints (1–5), and
- `rejected` — list of `(value, reason)` pairs. Use a `clean_likert(n)` that **raises** `ValueError` for out-of-range or non-ints.

Task 3 — Write a test

Put `clean_likert` in `clean.py` and write `test_clean.py` with pytest: one passing case and one `pytest.raises(ValueError)` case. Run `pytest`.

Task 4 — Discuss

Why is `except:` (bare) dangerous? Give one error it would hide that you'd rather see.

Bonus — Pythonic idiom drill

Cover the `# ->` answers, predict each line, then run.

```
class LikertError(ValueError):       # your own exception type
    pass
print(issubclass(LikertError, ValueError))   # -> True  (so `except ValueError` still
catches it)

try:
    assert 1 == 2, "values differ"   # assert: cheap internal sanity check
except AssertionError as e:
    print(e)                         # -> values differ
```

Solutions

```
def safe_int(value):
    try:
        return int(value)
    except (ValueError, TypeError):
        return None

def clean_likert(n):
    if isinstance(n, bool) or not isinstance(n, int):
        raise ValueError(f"{n!r} not an int")
    if not 1 <= n <= 5:
        raise ValueError(f"{n} outside 1–5")
    return n

raw = ["5","3","N/A","7","","1","two","4"]
clean, rejected = [], []
for r in raw:
    try:
        clean.append(clean_likert(safe_int(r)))
    except ValueError as e:
        rejected.append((r, str(e)))
print(clean)      # [5, 3, 1, 4]
print(rejected)   # [('N/A', ...), ('7', ...), ('', ...), ('two', ...)]
```

```
# test_clean.py
import pytest
from clean import clean_likert
def test_ok():   assert clean_likert(3) == 3
def test_bad():
    with pytest.raises(ValueError):
        clean_likert(9)
```

Task 4: a bare `except:` also catches `KeyboardInterrupt` (Ctrl+C) and `NameError` from your own typos — so a misspelled variable would be silently swallowed instead of showing you the bug. Always catch the specific exception you expect.

Part B

Files, Libraries & Research Data

Opening files with `with`

```
with open("notes.txt") as f:
    text = f.read()
# file auto-closes here, even if the code crashes
```

`with` = a context manager: sets up and tears down the resource for you. Always prefer it to a bare `open()` / `close()`.

File modes (mind the trap)

Mode	Meaning
`"r"`	read (default)
`"w"`	write — **truncates the file to empty first!**
`"a"`	append
`"r+"`	read + write

⚠ Open the wrong file with `"w"` → its contents are gone.

Reading text

```
with open("notes.txt") as f:
    whole = f.read()           # one big string
    # or
    for line in f:             # line by line (memory-friendly)
        print(line.rstrip())
```

⚠ A file object is exhausted after one pass — re-open to read again.

CSV in, as dicts □

```
import csv
with open("students.csv", newline="") as f:
    for row in csv.DictReader(f):
        print(row["name"], row["score"])   # row is a dict keyed by header
```

`csv.DictReader` turns each row into a dict — your "list of dicts" dataset from Session 4. (`newline=""` avoids blank rows on Windows.)

CSV out

```
with open("summary.csv", "w", newline="") as f:
    w = csv.DictWriter(f, fieldnames=["name", "score"])
    w.writeheader()
    w.writerow({"name": "Ana", "score": 91})
```

Libraries a researcher reaches for

```
import statistics
statistics.mean(xs); statistics.median(xs); statistics.stdev(xs)

import random
random.choice(xs); random.randint(1, 6); random.shuffle(xs)

from datetime import date
date.today()

from pathlib import Path
Path("students.csv").exists()
```

`pip install <package>` for third-party libs.

The pandas teaser (your next course)

```
import pandas as pd
df = pd.read_csv("students.csv")
df["score"].describe()      # count, mean, std, min, quartiles, max
df.groupby("major")["score"].mean()
```

Everything you did by hand today — in three lines. We learned the fundamentals *underneath* it first.

Your turn

`examples/session-04/practice.md` (uses `survey.csv`):

1. Read `students.csv`; print class mean with `statistics.mean`.
2. Compute per-item survey means; write `survey_summary.csv`.

Traps recap

- `"w"` silently overwrites — be sure of the filename.
- `csv` module → open with `newline=""`.
- Files exhaust after one read; re-open to re-read.
- Specify `encoding="utf-8"` for non-ASCII text.

Summary

You can load, summarize, and write real research data. **Next:** Regular expressions, modules & OOP.

Practice — Part B

Files provided: `students.csv`, `survey.csv`.

Task 1 — Read & summarize students

Read `students.csv` with `csv.DictReader`. Remember the values are **strings** — convert `score` to `int`. Print the class mean and median with the `statistics` module.

Task 2 — Mean by major

Build `{major: mean_score}`. (Hint: `dict.setdefault(key, []).append(...)`.)

Task 3 — Clean & summarize the survey

`survey.csv` has `"N/A"` and blanks in numeric columns. For each `q*` item, compute the mean of the **valid** values only, and how many were valid. Write `survey_summary.csv` with columns `item,mean,n_valid`.

Task 4 — pandas teaser (optional)

If `pandas` is installed: `pd.read_csv("students.csv")["score"].describe()`. Compare the mean to your hand-computed one.

Trap check

What happens if you accidentally open `students.csv` with mode `"w"` before reading it?

Bonus — Pythonic idiom drill

Cover the `# ->` answers, predict each line, then run.

```
import json
s = json.dumps({"n": 3, "ok": True})
print(s)                            # -> {"n": 3, "ok": true}   (Python True -> JSON true)
print(json.loads(s)["ok"])          # -> True                   (and back to a Python
bool)
```

Solutions

See `demo.py` in this folder — it implements Tasks 1–3 exactly. Key lines:

```
scores = [int(s["score"]) for s in students]     # convert strings!
statistics.mean(scores)                           # 75.5

by_major = {}
for s in students:
    by_major.setdefault(s["major"], []).append(int(s["score"]))

def to_int(x):
    try: return int(x)
    except (ValueError, TypeError): return None   # handles N/A and ""
```

Trap: opening with `"w"` **truncates the file to empty immediately** — your data is gone before you ever read it. Use `"r"` (the default) to read.

Traps — predict, then check

Cover the result, predict each one, then check yourself.

1.

```
int('3.0')
```

Most people expect `3` — the result is `ValueError: invalid literal for int() with base 10: '3.0'`. `int()` parses INTEGER text only; '3.0' is not a valid int literal. Use `int(float('3.0'))`.

2.

```
round(2.5)
```

Most people expect `3` — the result is `2`. Python uses banker's rounding (round half to EVEN): `round(2.5) == 2` but `round(3.5) == 4`. Watch this when summarising data.

3.

```
0.1 + 0.2 + 0.3 == 0.6
```

Most people expect `True` — the result is `False`. Float error accumulates when you total measurements: 0.1+0.2+0.3 is 0.6000000000000001. Compare sums with `math.isclose`, or round for display.

4.

```
int('1_000')
```

Most people expect `ValueError` — the result is `1000`. Python accepts underscores as digit separators, even inside `int()` text.

Session 5

Regular Expressions, Modules & OOP

Learn Python — a two-hour session, in two halves.

Part A: Regular Expressions & Text Cleaning · **Part B:** Modules, OOP & the Pythonic Toolkit

Part A

Regular Expressions & Text Cleaning

Why a researcher cares

- Validate IDs, emails, dates before they pollute your data.
- Extract structured bits from free text (codes, names, numbers).
- Clean and normalize open-ended survey responses.
- A first pass at **qualitative coding** (find every response matching a pattern).

 Like search-and-filter over a corpus — but it matches *form*, not meaning.

Always use raw strings

```
import re
re.search(r"\d+", "id 42")     # r"..." = raw string
```

Without `r"..."`, Python eats the backslashes (`\d` → error/garbage). **Rule:** every regex pattern is a raw string.

The survival tokens

Token	Matches
`.`	**any** char (except newline)
`\d \w \s`	digit / word-char / whitespace
`\D \W \S`	the negations
`+ * ?`	1+, 0+, 0-or-1
`{m}` `{m,n}`	exactly m / between m and n
`^ $`	start / end of string
`[abc]` `[^abc]`	any in set / none in set
`(...)`	capture group
`a|b`	a or b

The `.` trap

```
re.search(r".", "a.b").group()    # 'a'  — ANY char, not a dot!
re.search(r"\.", "a.b").group()   # '.'  — escape it for a literal dot
```

Escape the specials when you mean them literally: `\. \^ \$ \* \+ \? \( \) \[ \] \{ \} \|`

The four functions you need

```
re.search(pattern, s)     # first match ANYWHERE -> match or None
re.fullmatch(pattern, s)  # the WHOLE string must match -> validation
re.findall(pattern, s)    # list of ALL matches
re.sub(pattern, repl, s)  # replace matches -> cleaning
```

`re.IGNORECASE` flag for case-insensitive: `re.search(p, s, re.IGNORECASE)`.

Validate (fullmatch anchors both ends)

```
def valid_university_email(addr):
    return re.fullmatch(r"\w+@\w+\.edu", addr) is not None

valid_university_email("ana@university.edu")   # True
valid_university_email("ana@gmail.com")        # False
valid_university_email("ana@@x.edu")           # False
```

Extract with capture groups

```
m = re.search(r"([A-Z]{2})(\d{4})", "Course ED1234 meets Tue")
m.group(0)   # "ED1234"  whole match
m.group(1)   # "ED"      dept
m.group(2)   # "1234"    number
```

`m` is `None` if nothing matched — check before `.group()`.

Clean & mine free text

```
re.sub(r"\s+", " ", messy).strip()        # collapse whitespace
re.findall(r"#(\w+)", "love #python #stats")   # ['python', 'stats']

from collections import Counter
Counter(re.findall(r"#(\w+)", corpus))    # theme frequencies
```

Reformat with groups: `re.sub(r"^(.+),\s*(.+)$", r"\2 \1", "Curie, Marie")` → `"Marie Curie"`.

When NOT to use regex

```
"a,b,c".split(",")     # simple split — no regex needed
"  hi  ".strip()       # trim — no regex needed
text.replace("X", "Y") # fixed substring — no regex needed
```

Regex shines for *variable* patterns. For fixed strings, plain methods read better.

Your turn

`examples/session-05/practice.md`:

1. Email validator. 2. Extract dept+number. 3. Collapse whitespace.
2. Count hashtags across responses. 5. Flip `"Last, First"`. 6. One case to use `.split()` instead.

Traps recap

- `.` matches **any** char — use `\.` for a literal dot.
- Forgetting `r"..."` breaks your backslashes.
- `re.search` returns `None` on no match — guard before `.group()`.
- Don't use regex where a string method is clearer.

Summary

You can validate, extract, and clean real-world text. **Next:** *Part B of this session* — modules, OOP & the Pythonic toolkit.

Practice — Part A

Always use raw strings `r"..."`. Predict each result before running.

Task 1 — Validate

Write `valid_university_email(addr)` returning `True` only for `something@something.edu`. Test: `"ana@university.edu"`, `"ana@gmail.com"`, `"a@b.edu.evil.com"`.

Task 2 — Extract with groups

From `"Course ED1234 meets Tue"`, pull the department (`ED`) and number (`1234`) using one regex with two capture groups.

Task 3 — Clean

Collapse all runs of whitespace in `" too much\t space "` to single spaces and trim.

Task 4 — Mine free text

From a list of open-ended responses, count how often each `#hashtag` appears (use `re.findall(r"#(\w+)", text)` and `collections.Counter`).

Task 5 — Reformat

Turn `"Curie, Marie"` into `"Marie Curie"` with a single regex + groups.

Task 6 — Judgment

Give one task where a plain string method (`.split()`, `.strip()`, `.replace()`) is the better, clearer choice than a regex.

Bonus — Pythonic idiom drill

Cover the `# ->` answers, predict each line, then run.

```
import re
m = re.search(r"(?P<year>\d{4})", "class of 2026")
print(m.group("year"), m.groupdict())   # -> 2026 {'year': '2026'}   (named groups)
print(re.split(r"\s*,\s*", "a, b ,c")) # -> ['a', 'b', 'c']        (split on commas +
spaces)
```

Solutions

See `demo.py` in this folder — it implements all six. Key lines:

```
re.fullmatch(r"\w+@\w+\.edu", addr) is not None      # 1 (fullmatch anchors both ends)
m = re.search(r"([A-Z]{2})(\d{4})", s); m.group(1), m.group(2)   # 2
re.sub(r"\s+", " ", messy).strip()                   # 3
from collections import Counter; Counter(re.findall(r"#(\w+)", text))   # 4
m = re.search(r"^(.+),\s*(.+)$", s); f"{m.group(2)} {m.group(1)}"      # 5
```

Task 6: splitting `"a,b,c"` on commas is just `"a,b,c".split(",")` — no regex needed. Reach for regex only when the pattern is genuinely variable (digits, optional parts, anchors).

```

```

Trap reminder: `.` matches **any** character — use `\.` for a literal dot, and never forget the `r"..."` prefix or your backslashes become Python escape sequences.

Part B

Modules, OOP & the Pythonic Toolkit

Modules: split your code into files

```
# grades.py
def letter_grade(score): ...
def class_average(scores): ...

# analysis.py
from grades import letter_grade, class_average
```

A `.py` file is a **module**. `import` reuses its functions elsewhere → no copy-paste.

The `__main__` guard

```
# grades.py
if __name__ == "__main__":
    print(letter_grade(85))   # runs ONLY when you execute grades.py directly
```

On `import grades`, `__name__` is `"grades"`, so the block is skipped. One file can be both a runnable script *and* an importable library.

OOP: model a domain entity

```
class Student:
    def __init__(self, name, gpa):   # constructor
        self.name = name
        self.gpa = gpa
    def __str__(self):               # how it prints
        return f"{self.name} ({self.gpa})"

ana = Student("Ana", 3.9)
print(ana)        # Ana (3.9)
```

 A class is an *operational definition*: the attributes + behaviors that "count" as a Student. `self` = "this particular student."

@property: validate on assignment

```
class Student:
    ...
    @property
    def gpa(self):
        return self._gpa
    @gpa.setter
    def gpa(self, value):
        if not 0 <= value <= 4:
            raise ValueError("gpa must be 0–4")
        self._gpa = value
```

`ana.gpa = 5.0` now raises — the object defends its own integrity.

Inheritance

```
class GradStudent(Student):
    def __init__(self, name, gpa, advisor):
        super().__init__(name, gpa)    # reuse parent setup
        self.advisor = advisor
    def __str__(self):
        return super().__str__() + f" — {self.advisor}"
```

`GradStudent` *is a* `Student` plus extra. `super()` calls the parent.

The Pythonic toolkit (recap tour)

```
[s.name for s in roster if s.gpa >= 2.0]   # comprehension (from S4)
list(map(lambda s: s.name.upper(), roster))# map: apply to all
list(filter(lambda s: s.gpa < 2.0, roster))# filter: keep matches
for i, s in enumerate(roster): ...          # index + item
for a, b in zip(names, scores): ...         # parallel
```

Generators & walrus

```
def gpas(students):
    for s in students:
        yield s.gpa          # one value at a time — low memory on big data

g = gpas(roster)
list(g)   # ?
list(g)   # ?  ← run it twice — what changes? (Traps below)

if (n := len(roster)) > 30:   # walrus := : assign + test in one step
    print(f"{n} students")
```

Your turn

`examples/session-05/practice.md`:

1. Import from `grades.py`. 2. Build the validating `Student` class.
2. Add `GradStudent` with `super()`. 4. Comprehension + `map` + `filter` + a generator.

Traps recap

- `self` is just "this instance" — not magic.
- A generator iterates **once**, then it's empty.
- The `__main__` guard keeps imported modules from running their demo code.
- Don't reach for a class when a function or dict will do.

Summary

You can structure code into modules and classes and write idiomatic Python. **Next (optional):** the capstone — put it all together.

Next: the capstone project.

Practice — Part B

Files in this folder: `grades.py` (a module you import), `demo.py` (worked example).

Task 1 — Use a module

From a new file, `from grades import letter_grade, class_average` and call both. Why does the `if __name__ == "__main__":` block in `grades.py` NOT run when you import it?

Task 2 — A class with a validating property

Build `Student(name, gpa)` with:

- `__str__` → `"Ana: 3.9 (Good)"`,
- `standing()` → `"Good"` if gpa ≥ 2.0 else `"Probation"`,
- a `@property` setter for `gpa` that raises `ValueError` outside 0–4. Prove the setter rejects `5.0`.

Task 3 — Inheritance

Add `GradStudent(Student)` that also stores an `advisor` and uses `super().__init__(...)`. Override `__str__` to append the advisor.

Task 4 — The Pythonic toolkit

Given a roster of `Student`s:

1. names in good standing (list comprehension),
2. uppercase names (`map`),
3. at-risk students (`filter`),
4. mean gpa via a **generator** that `yield`s each gpa — then show the generator is empty on a second pass.

Bonus — Pythonic idiom drill

Cover the `# ->` answers, predict each line, then run.

```
from dataclasses import dataclass

@dataclass                          # auto __init__, __repr__, __eq__
class Point:
    x: int
    y: int
print(Point(1, 2), Point(1, 2) == Point(1, 2))   # -> Point(x=1, y=2) True

def head(v):
    match v:                        # structural pattern matching (3.10+)
        case [first, *_]: return first
        case _: return None
print(head([9, 8]), head(5))        # -> 9 None
```

Solutions

See `demo.py` — it implements Tasks 2–4. Key points:

```
@property
def gpa(self): return self._gpa
@gpa.setter
def gpa(self, v):
    if not 0 <= v <= 4: raise ValueError(...)
    self._gpa = v

class GradStudent(Student):
    def __init__(self, name, gpa, advisor):
        super().__init__(name, gpa)
        self.advisor = advisor

[s.name for s in roster if s.gpa >= 2.0]          # comprehension
list(map(lambda s: s.name.upper(), roster))        # map
[s.name for s in filter(lambda s: s.gpa < 2.0, roster)]   # filter
def gpas(rs):
    for s in rs: yield s.gpa                       # generator (exhausts after one pass)
```

Task 1: the `__name__` guard is only `"__main__"` when the file is **run directly**; on `import` its `__name__` is `"grades"`, so the demo block is skipped — that's how a file can be both a runnable script and an importable module.

Traps — predict, then check

Cover the result, predict each one, then check yourself.

1.

```
gen = (n * n for n in range(3))
first = list(gen)
list(gen)
```

Most people expect `[0, 1, 4] again` — the result is `[]`. A generator is one-shot: after the first full pass it is exhausted. Rebuild it, or store a list if you need it twice.

2.

```
from dataclasses import dataclass

@dataclass
class Pt:
    x: int

p1 = Pt(1)
p2 = Pt(1)
p1 == p2
```

Most people expect `False — two different objects` — the result is `True`. `@dataclass` writes an `__eq__` that compares fields, so equal-valued instances are `==` (even though `p1 is p2` is False).

3.

```
import re
m = re.match(r'<(.+)>', '<a><b>')
m.group(1)
```

Most people expect `'a'` — the result is `'a><b'`. `+` is greedy — it grabs as much as it can. Use `+?` for the shortest match: `r'<(.+?)>'`.

4.

```
import re
bool(re.match(r'\d+', 'abc123'))
```

Most people expect `True — there are digits` — the result is `False`. `re.match` anchors at the START of the string. Use `re.search` to find a match anywhere.

5.

```
class Dog:
    tricks = []
    def teach(self, t):
        self.tricks.append(t)

a = Dog()
b = Dog()
a.teach('sit')
b.tricks
```

Most people expect `[] — b is a different dog` — the result is `['sit']`. `tricks` is a CLASS variable shared by every instance. Give each its own in `__init__`: `self.tricks = []`.

Setup & Tools

Everything you need to run Python on your own machine — plus the tools (and AI habits) that make learning this material faster.

Fastest start: no install at all

Every example on this site runs **in your browser** — just press **Run**. Nothing to install, great for the first sessions. When you want to work on *your own files*, install Python locally with the steps below.

Run the course as Jupyter notebooks

Every session is also a **Jupyter notebook** — the same examples and practice as a runnable, cell-by-cell document. Three ways to use them; pick whatever fits:

- **In your browser, no install** — open the live JupyterLite workspace: https://yangnei.github.io/learn-python/jupyter/lab/index.html. It's real Jupyter running entirely in your browser; your edits are saved locally in the browser, nothing is uploaded. Each session page also has a **Run in browser** button that opens that session's notebook directly.
- **Google Colab** — each session page has an **Open in Colab** button (free, runs in Google's cloud, needs a Google account). Handy when you want to save to Drive or do heavier data work.
- **Local Jupyter** — once Python is installed (below): `pip install jupyterlab`, then run `jupyter lab` and open the `.ipynb` you saved with a session page's **Download .ipynb** button.

How to drive a notebook: click a cell and press **Shift + Enter** to run it and move to the next. Predict the output before you run — the surprise is the lesson — then edit a cell and re-run to experiment.

Autocomplete & libraries: press **Tab** to complete a name and **Shift + Tab** to pop up a function's help. To add a package, run `%pip install <name>` *in a cell* (e.g. `%pip install pandas`). In the browser (JupyterLite) only Pyodide-compatible packages work — the big ones like `pandas`, `numpy`, and `matplotlib` do — and the install lasts for the session, so re-run that cell after a kernel restart. In local Jupyter or Colab, `%pip install` is permanent for that environment.

Install Python (pick your OS)

Windows

- Official installer: https://www.python.org/downloads/windows/ — on the first screen, **check "Add python.exe to PATH"**, then *Install Now*.
- (Alternative) Microsoft Store → search **Python 3.12**.
- Check it worked — open **PowerShell** and run: `python --version`

- Official guide: https://docs.python.org/3/using/windows.html

macOS

- Official installer: https://www.python.org/downloads/macos/
- (Alternative) Homebrew: `brew install python`
- Check it worked — open **Terminal** and run: `python3 --version`
- Official guide: https://docs.python.org/3/using/mac.html

Linux

- Usually already installed. If not: `sudo apt install python3 python3-pip` (Debian/Ubuntu) or `sudo dnf install python3` (Fedora).
- Check it worked: `python3 --version`
- Official guide: https://docs.python.org/3/using/unix.html

An editor: install **VS Code** (https://code.visualstudio.com/) and its **Python extension** (by Microsoft). Step-by-step: https://code.visualstudio.com/docs/python/python-tutorial. Prefer something lighter? **Thonny** (https://thonny.org/) is a beginner IDE with a built-in variable viewer and debugger.

Run a script: save your code as `name.py`, then in a terminal run `python name.py` (Windows) or `python3 name.py` (macOS/Linux).

Install a package (e.g. for the Session 4 pandas teaser): `pip install pandas` (or `pip3`).

Visualize what your code does — Python Tutor

https://pythontutor.com/ — paste code and **step through it line by line**, watching variables, the call stack, and which names point at which objects. It's the single best tool for the things this course keeps warning you about:

- **aliasing** — see `b = a` make both names point at the *same* list (Session 2),
- the **mutable-default-argument** bug (Session 3),
- the **recursion call stack** building up and unwinding (Session 3).

Whenever you think *"wait, why did that happen?"* — drop the snippet into Python Tutor.

Use AI to learn (without it doing the work for you)

An AI assistant (Claude, etc.) is a huge accelerator *if you ask it to explain rather than to write*. Type every solution yourself. Prompts that fit this course:

- **"Summarize this doc page for a beginner, just the 5 things I need: \<paste text or URL>"** — turn a dense reference page into something usable.
- **"Explain this error and its most likely cause, and point me to the line: \<paste the traceback>"**

- **"Predict what this prints, then explain why: \<code>"** — then run it and compare. That predict-then-run habit *is* the method this course is built on.
- **"Review my function for the traps in this course — identity vs value, aliasing, mutable defaults, off-by-one — but don't rewrite it for me."**
- **"Give me 3 harder practice tasks like this one, with the solutions hidden."**

Rule of thumb: **ask it to explain, not to do.** If it hands you code you can't read line by line, ask it to slow down and walk you through it.

Other handy tools

- **regex101.com** (https://regex101.com/) — build and *explain* regular expressions interactively, with a live match view (Session 5). Set the flavor to **Python**.
- **Official docs** — the Python Tutorial and the Library Reference. Bookmark both.
- **Google Colab** (https://colab.research.google.com/) — run Python notebooks in the browser, no install; a natural next step once you start doing real data work.
- This site's **Traps & Gotchas** sheet — keep it open the whole course.

Python Traps & Gotchas — The Master Cheat Sheet

> The quirks that bite beginners (and plenty of experts). Every behavior below was run and verified on CPython 3.11+. Read the **Surprise** column, cover the **Why/Fix**, and predict. Keep this open for the whole course.

---

1 — Equality `==` vs Identity `is` *(Session 1)*

Expression	Result	
`a = [1,2]; b = [1,2]; a == b`	`True`	same **value**
`a = [1,2]; b = [1,2]; a is b`	`False`	different **objects** in memory
`a = [1,2]; b = a; a is b`	`True`	`b` is the *same* object (an alias)

- `==` asks "same value?" — this is what you almost always want.
- `is` asks "same object?" — use it **only** for `None`, `True`, `False`: `python if x is None: # ✅ correct if x == None: # ⚠ works, but not idiomatic — use `is`
`
- **Identity wrinkle (do NOT rely on this):** CPython caches small integers (−5 to 256) and interns some strings, so `a = 256; b = 256; a is b` is `True` but at `257` it may be `False`. This is an implementation detail. **For value, always use `==`.**

 Bridge: two students with the same GPA are `==`; the same student is `is`.

---

2 — Booleans ARE integers *(Session 1)*

```
True == 1            # True   (bool is a subclass of int)
False == 0           # True
5 + True             # 6      yes, you can do arithmetic on bools
sum([True, False, True])   # 2   ← counts the Trues
isinstance(True, int)      # True
type(True) is int          # False  (its type is bool, a subtype)
```

- **Fix / use it well:** `sum(conditions)` is the Pythonic way to *count* how many are true.
- **Trap:** `True is 1` is `False` (different objects) even though `True == 1`. Compare with `==`.

 Bridge: this is dummy coding (1/0) built into the language — summing flags counts cases.

---

3 — Float precision *(Session 1)*

```
0.1 + 0.2           # 0.30000000000000004
0.1 + 0.2 == 0.3    # False  😱
```

- **Why:** decimals are stored in binary; most can't be represented exactly. Not a Python bug — every language does this.
- **Fix:** `python import math math.isclose(0.1 + 0.2, 0.3) # True ← compare floats with a tolerance round(0.1 + 0.2, 2) == 0.3 # True ← or round for display/compare from decimal import Decimal # exact decimal arithmetic when you truly need it (money, grades) Decimal("0.1") + Decimal("0.2") # Decimal('0.3')`
- **Rule:** never test computed floats with `==`. Use `math.isclose` (or `round`).

 Bridge: you already never test two measured scores for exact equality — same instinct, new cause (binary storage, not measurement error).

---

4 — Comparing across types *(Session 1)*

```
5 == "5"     # False   (different types → not equal, but NO error)
5 == 5.0     # True    (int vs float compare by numeric value)
5 > "5"      # ▫ TypeError: '>' not supported between 'int' and 'str'
```

- **Equality (`==` / `!=`)** across unrelated types → just `False`, never crashes.
- **Ordering (`<`, `>`, `<=`, `>=`)** across incompatible types → **`TypeError`**.
- **Fix:** convert first. `int("5") == 5` → `True`. Guard with `isinstance` before ordering.

 Bridge: the computer can equate "are these the same?" across types, but can't *rank* text against numbers — it has no shared scale.

---

5 — Sequences compare element-by-element *(Session 1 / 2)*

```
[1, 2] == [1, 2]     # True
[1, 2] == (1, 2)     # False   ← list vs tuple: different types, never equal
(1, 2) < (1, 3)      # True    ← compares position by position (lexicographic)
"apple" < "banana"   # True    ← strings compare char by char (alphabetical-ish)
[1, 2] < [1, 2, 3]   # True    ← prefix is "less than"
```

- **Trap:** a `list` and a `tuple` with identical contents are **never** `==` — type matters.
- Lists/tuples/strings compare left-to-right; the first differing element decides.

---

6 — Truthiness (what counts as False) *(Session 1 / 2)*

```
# These are all "falsy":
bool(0) bool(0.0) bool("") bool([]) bool({}) bool(set()) bool(None)   # all False
bool("0")   # True!  ← a non-empty string is truthy, even "0"
bool([0])   # True   ← a non-empty list is truthy, even if it holds a 0
```

- **Fix / idiom:** check emptiness directly — `if scores:` not `if len(scores) > 0:`; `if name:` not `if name != "":`.
- **Trap:** `"0"` (string) and `"False"` (string) are **truthy**. Convert user input before testing.

---

7 — `and` / `or` return an operand, not a bool *(Session 2)*

```
5 and 0      # 0      (and → first falsy, or last value)
0 or "hi"    # "hi"   (or → first truthy, or last value)
"" or "N/A"  # "N/A"  ← handy default-value idiom
```

- **Use it well:** `name = user_input or "Anonymous"` supplies a default.
- **Trap:** don't write `if x == True`. Just `if x:`. And `and` / `or` short-circuit — the right side may never run, so don't hide side effects there.

---

8 — Mutable default arguments *(Session 3)* — the famous one

```
def add_student(name, roster=[]):     # ✖ DANGER
    roster.append(name)
    return roster

add_student("Ana")     # ['Ana']
add_student("Ben")     # ['Ana', 'Ben']  😱  the default list PERSISTS across calls
```

- **Why:** the default `[]` is created **once**, when the function is *defined*, and reused every call.
- **Fix:** `python def add_student(name, roster=None): # ✅ if roster is None: roster = [] roster.append(name) return roster`
- **Rule:** never use a mutable default (`[]` , `{}` , `set()`). Use `None` and create inside.

---

9 — Aliasing: variables are labels, not boxes *(Session 2)*

```
a = [1, 2, 3]
b = a            # b is the SAME list, not a copy
a.append(4)
b                # [1, 2, 3, 4]  😱  b changed too

b = a.copy()     # ✅ now b is an independent (shallow) copy
import copy
b = copy.deepcopy(a)   # ✅ independent even for nested lists/dicts
```

- **Trap:** `grid = [[0] * 3] * 3` makes **three references to one row** — editing `grid[0][0]` changes all rows. Use `[[0]*3 for _ in range(3)]`.
- **Rule:** assignment never copies. `=` binds another name to the same object.

□ *Bridge:* a variable is a sticky label on an object, not a container holding a value.

---

10 — `type()` vs `isinstance()` *(Session 1)*

```
isinstance(x, int)              # ✅ Pythonic; respects inheritance
isinstance(x, (int, float))     # ✅ "is x any kind of number?"
type(x) is int                  # exact type only; rarely what you want
type(x) == int                  # works but use `is` for type identity
```

- **Trap:** `isinstance(True, int)` is `True` (bool is a subtype of int). If you must exclude bools, check `type(x) is int` or `isinstance(x, int) and not isinstance(x, bool)`.

---

11 — Integer vs float division *(Session 1)*

```
7 / 2      # 3.5   true division ALWAYS returns a float
7 // 2     # 3     floor division (rounds toward −∞)
-7 // 2    # -4    ← floors, doesn't truncate toward zero!
7 % 2      # 1     modulo (remainder) — use % 2 to test even/odd
7 / 0      # ▫ ZeroDivisionError
```

- **Trap:** `/` gives a float even when the result is whole (`4 / 2` is `2.0`). Use `//` for integer results.

---

12 — Strings are immutable; some methods *return*, don't mutate *(Session 1 / 5)*

```
s = "  Hello  "
s.strip()          # "Hello"   ← returns a NEW string
s                  # "  Hello  "  ← s is UNCHANGED
s = s.strip()      # ✅ reassign to keep the result
```

- **Trap:** `s.strip()` / `s.replace(...)` / `s.upper()` do nothing to `s` unless you reassign.
- Contrast: list methods like `.append()` / `.sort()` mutate **in place** and return `None`: `python nums = [3, 1, 2] nums = nums.sort() # ✖ now nums is None! .sort() returns None nums.sort() # ✔ sorts in place; use sorted(nums) to get a new list`

13 — `range` excludes the stop; off-by-one *(Session 2)*

```
list(range(1, 5))      # [1, 2, 3, 4]   ← 5 is NOT included
list(range(5))         # [0, 1, 2, 3, 4]
list(range(0, 10, 2))  # [0, 2, 4, 6, 8]
```

- **Rule:** `range(a, b)` is the half-open interval `[a, b)`.

14 — Modifying a list while iterating it *(Session 2)*

```
xs = [1, 2, 3, 4]
for x in xs:
    if x % 2 == 0:
        xs.remove(x)     # ✖ skips elements / unpredictable
# ✔ iterate a copy, or build a new list:
xs = [x for x in xs if x % 2 != 0]
```

15 — Files: `"w"` overwrites; cursor exhausts *(Session 4)*

```
open("data.csv", "w")    # ! TRUNCATES the file to empty immediately
open("data.csv", "a")    # append
open("data.csv", "r")    # read (default)

with open("data.csv") as f:
    rows = f.readlines()      # read once
    again = f.readlines()     # [] — cursor is at end of file now
```

- Always prefer `with open(...) as f:` — it closes the file even if the code crashes.
- With the `csv` module on Windows, open with `newline=""` to avoid blank rows.

16 — Regex: `.` matches *anything*; use raw strings *(Session 5)*

```
import re
re.search(r"\d+", "id 42")     # use r"..." so \d isn't a Python escape
re.search(".", "a.b")          # the "." matches "a", not just the dot!
re.search(r"\.", "a.b")        # use \. for a literal dot
```

- **Rule:** always write patterns as raw strings `r"..."`. Escape `. ^ $ * + ? ( ) [ ] { } | \`.

Quick "predict-then-run" self-test (cover the answers)

```
print(True + True)            # 2
print(3 == 3.0)               # True
print(0.1 + 0.2 == 0.3)       # False
print(5 == "5")               # False
print([1,2] == (1,2))         # False
print(bool("0"))              # True
print(7 // 2, -7 // 2)        # 3 -4
x = [1]; y = x; x.append(2); print(y)   # [1, 2]
```

If you can explain *why* for all eight, you've got the fundamentals most beginners miss.

Python Quick Reference — Syntax You'll Forget

> The "how do I write that again?" sheet. Grouped by session.

I/O & types *(S1)*

```
name = input("Name: ")          # ALWAYS returns a str
age  = int(input("Age: "))      # convert immediately
print("Hi", name, age)          # space-separated
print(f"Hi {name}, age {age}")  # f-string (preferred)
print(f"{score:.2f}")           # 2 decimals
print(f"{count:,}")             # thousands separator: 1,234
print("no newline", end="")     # suppress the trailing \n
int("42"), float("3.14"), str(42), bool(0)   # casts
type(x)                          # what type is x?
```

f-string formatting mini-table

Spec	Example	Output
`:.2f`	`f"{3.14159:.2f}"`	`3.14`
`:,`	`f"{1234567:,}"`	`1,234,567`
`:>8`	`f"{'hi':>8}"`	`hi` (right-align)
`:<8`	`f"{'hi':<8}"`	`hi` (left-align)
`:^8`	`f"{'hi':^8}"`	`hi` (center)
`:.1%`	`f"{0.873:.1%}"`	`87.3%`

Conditionals *(S2)*

```
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
else:
    grade = "F"

90 <= score <= 100              # chained comparison
grade = "pass" if score >= 60 else "fail"   # ternary

match command:                   # Python 3.10+
    case "start": ...
    case "stop" | "halt": ...    # multiple values
    case _: ...                  # default
```

Loops *(S2)*

```
for i in range(5): ...                  # 0..4
for x in items: ...                     # each element
for i, x in enumerate(items): ...       # index + element
for a, b in zip(names, scores): ...     # walk two lists together
while condition: ...
while True:                             # validation loop
    x = input("...")
    if valid(x):
        break
# break / continue control flow
```

Sequences & slicing *(S2)*

```
xs = [1, 2, 3]; xs.append(4); xs[0]; xs[-1]   # last
xs[1:3]      # [2, 3]   (start inclusive, stop exclusive)
xs[:2]       # first two
xs[::-1]     # reversed
t = (1, 2)                      # tuple (immutable)
d = {"name": "Ana", "gpa": 3.9}  # dict
d["name"]; d.get("missing", 0)   # access; safe access w/ default
d.keys(); d.values(); d.items()
s = {1, 2, 2, 3}                # set -> {1, 2, 3}  (unique)
```

Comprehensions *(S2/S5)*

```
[x*2 for x in xs]                       # list
[x for x in xs if x > 0]                # with filter
{name: 0 for name in names}             # dict
{x % 3 for x in xs}                     # set
(x*x for x in xs)                       # generator (lazy)
```

Sorting *(S2)*

```
sorted(xs)                              # new sorted list
sorted(xs, reverse=True)
sorted(students, key=lambda s: s["gpa"])          # by a field
sorted(students, key=lambda s: s["gpa"], reverse=True)
xs.sort()                               # in place (returns None!)
```

Functions *(S3)*

```
def avg(nums: list[float]) -> float:
    """Return the mean of nums."""      # docstring
    return sum(nums) / len(nums)

def greet(name, greeting="Hello"): ...  # default arg (NOT mutable!)
def total(*args): ...                   # any number of positionals -> tuple
def config(**kwargs): ...               # any number of keywords -> dict
func(*my_list)                          # unpack list into args
func(**my_dict)                         # unpack dict into kwargs
```

Exceptions *(S4)*

```
try:
    n = int(value)
except ValueError:
    n = None
except (KeyError, IndexError) as e:
    print(e)
else:
    print("ok")          # ran only if no exception
finally:
    print("always")      # cleanup, always runs

raise ValueError("score must be 1–5")
assert n > 0, "n must be positive"
```

Files & CSV *(S4)*

```
with open("f.txt") as f:          # read, auto-closes
    text = f.read()
with open("f.txt", "w") as f:     # write (OVERWRITES)
    f.write("line\n")

import csv
with open("data.csv", newline="") as f:
    for row in csv.DictReader(f):    # row is a dict keyed by header
        print(row["name"])

with open("out.csv", "w", newline="") as f:
    w = csv.DictWriter(f, fieldnames=["name", "gpa"])
    w.writeheader()
    w.writerow({"name": "Ana", "gpa": 3.9})
```

Handy stdlib *(S4)*

```
import statistics; statistics.mean(xs); statistics.median(xs); statistics.stdev(xs)
import random; random.choice(xs); random.randint(1, 6); random.shuffle(xs)
from datetime import date; date.today(); date(2026, 6, 26)
from pathlib import Path; Path("data.csv").exists()
import json; json.dumps(obj, indent=2); json.loads(text)
```

Regex *(S5)*

```
import re
re.search(r"pattern", text)      # first match anywhere (or None)
re.fullmatch(r"\d{5}", zip)      # whole string must match
re.sub(r"\s+", " ", text)        # collapse whitespace
m = re.search(r"(\w+)@(\w+)", s)
m.group(1)                        # first capture group
```

Token	Means
`.`	any char (except newline)
`\d \w \s`	digit / word-char / whitespace
`+ * ?`	1+, 0+, 0-or-1
`{m,n}`	between m and n
`^ $`	start / end
`[...] [^...]`	set / not-in-set
`(...)`	capture group
`a|b`	a or b

Classes *(S5)*

```
class Student:
    def __init__(self, name, gpa):
        self.name = name
        self._gpa = gpa
    def __str__(self):
        return f"{self.name} ({self._gpa})"
    @property
    def gpa(self):
        return self._gpa
    @gpa.setter
    def gpa(self, value):
        if not 0 <= value <= 4:
            raise ValueError("gpa out of range")
        self._gpa = value

s = Student("Ana", 3.9)
print(s)            # uses __str__
```

The program skeleton *(every script)*

```
def main():
    ...

def helper():
    ...

if __name__ == "__main__":
    main()
```

Glossary — Plain-Language Definitions

For a learner who knows research but not code. Each term: what it is, in one breath.

- **Argument** — a value you hand to a function when you call it. `int("5")` — `"5"` is the argument.
- **Parameter** — the named slot in a function definition that receives an argument. `def f(x):` — `x` is the parameter.
- **Variable** — a *name* (label) pointing at an object. Not a box; a sticky note.
- **Object** — any value Python stores: a number, string, list, function, even a type. "Everything is an object."
- **Type** — the kind of an object: `int`, `float`, `str`, `bool`, `list`, `dict`, `set`, `tuple`, `NoneType`.
- **Dynamic typing** — variables aren't locked to one type; the *object* has a type, the *name* doesn't.
- **`int` / `float`** — whole number / decimal number. `/` always makes a float; `//` floors to int.
- **`str`** — text. Immutable (can't change in place). Written in quotes.
- **`bool`** — `True` / `False`. Technically a kind of `int` (`True == 1`).
- **`None`** — "no value." Returned by functions that don't `return` anything. Test with `is None`.
- **f-string** — `f"...{expr}..."` — a string with `{}` placeholders filled by Python.
- **Equality (`==`)** — "same value?" The comparison you usually want.
- **Identity (`is`)** — "literally the same object in memory?" Use only for `None` / `True` / `False`.
- **Truthy / Falsy** — any value used in an `if` counts as true unless it's `0`, `""`, `[]`, `{}`, `set()`, or `None`.
- **Mutable / Immutable** — can it be changed in place? `list` / `dict` / `set` yes; `int` / `str` / `tuple` no.
- **Aliasing** — two names pointing at the *same* mutable object; changing one shows in the other.
- **Slicing** — taking a sub-sequence: `xs[start:stop:step]`, stop excluded.
- **Comprehension** — a one-line way to build a list/dict/set from a loop: `[x*2 for x in xs]`.
- **Function** — a reusable named block of code; takes inputs, optionally `return`s an output.
- **`return` vs `print`** — `return` hands a value back to the *caller*; `print` just shows text to the screen.
- **Scope** — where a name is visible. Python looks Local → Enclosing → Global → Built-in (LEGB).
- **`*args` / `**kwargs`** — collect "any number of" positional args (as a tuple) / keyword args (as a dict).
- **Type hint** — optional annotation of expected types (`def f(x: int) -> str:`). Documentation, **not** enforced at runtime.
- **Exception** — an error raised at runtime (e.g., `ValueError`). Handle with `try` / `except`.
- **`raise`** — deliberately throw an exception. **EAFP** — "easier to ask forgiveness than permission": try it, catch failure.
- **`assert`** — a developer sanity-check that errors if false. Not for validating user input (can be turned off).
- **Module** — a `.py` file you can `import`. **Library / package** — a bundle of modules (install via `pip`).
- **Standard library** — modules that ship with Python (`csv`, `statistics`, `random`, `re`, `json`, `datetime`, `pathlib`).
- **Context manager (`with`)** — a block that sets up and tears down a resource automatically (e.g., closes a file).
- **CSV** — comma-separated values; a plain-text table. `csv.DictReader` turns each row into a dict.

- **Class / Object (OOP)** — a blueprint (`class`) and the things built from it (instances). Bundles data + behavior.
- **`self`** — inside a class, "this particular instance."
- **`__init__`** — the setup method run when you create an instance. **Dunder** = "double underscore" method.
- **`@property`** — makes a method look like an attribute; lets you validate on get/set.
- **Generator / `yield`** — produces values one at a time, lazily; saves memory on big data. Can be iterated once.
- **`lambda`** — a tiny anonymous function, often used as a `key=` for sorting.
- **Regex (regular expression)** — a pattern language for matching/extracting text (`re` module).
- **Traceback** — the error report Python prints when something fails. **Read the last line first.**
- **REPL** — the interactive `>>>` prompt where you type one line at a time.

Per-Session Quizzes (with answer keys)

Each quiz is ~6 questions (about three per half), ~8 minutes, given at the end of the session. Every question maps to a session objective. **Teacher:** ask the student to *predict* code output before they run it — the surprise is the assessment. Answers are at the end of each session block.

Session 1 — Running Python, Types & the Type Traps

Part A — Variables & Types

1. What type does `input("x: ")` return, always?
2. What does `"5" + "3"` evaluate to? How do you get `8`? And what is `int(3.9)`?

Part B — The Dynamic-Typing Traps

3. `a = [1,2]; b = [1,2]` — is `a == b`? is `a is b`? Why?
4. What is `True + True`? Why?
5. Is `0.1 + 0.2 == 0.3` True or False? How should you compare them?
6. What does `5 == "5"` give? What about `5 > "5"`?

Answers: 1. `str`. 2. `"53"`; use `int("5") + int("3")`; `int(3.9)` is `3` (truncates).

3. `==` True (same value), `is` False (different objects). 4. `2` (bool is a subclass of int).
4. False; use `math.isclose(0.1+0.2, 0.3)` (or round). 6. `False`; `5 > "5"` raises `TypeError` (can't order int vs str).

Session 2 — Control Flow & Data Structures

Part A — Conditionals & Loops

1. Rewrite `if x >= 90 and x < 100:` using a chained comparison.
2. What is `5 and 0`? What is `0 or "hi"`? Why aren't they `True`/`False`?
3. What does `list(range(1, 5))` produce?

Part B — Data Structures

4. Which are mutable: list, tuple, dict, set?
5. `a = [1,2]; b = a; a.append(3)` — what is `b`? Why?
6. (Practical) Write a one-line dict comprehension mapping each name in `names` to `0`.

Answers: 1. `if 90 <= x < 100:`. 2. `0` and `"hi"` — `and`/`or` return an operand, not a bool.

3. `[1, 2, 3, 4]` (5 excluded). 4. list, dict, set are mutable; tuple is not. 5. `[1, 2, 3]` — `b` is an alias for the same list. 6. `{n: 0 for n in names}`.

Session 3 — Functions, Scope & Recursion

Part A — Functions & Scope

1. What's the difference between `return` and `print`?
2. Why is `def f(x, items=[])` dangerous? What's the fix?
3. What does a function with no `return` statement return? Are type hints enforced at runtime?

Part B — Recursion

4. What two parts must every recursive function have?
5. What error comes from a base case that's never reached, and why doesn't Python just keep going?
6. Why is recursion a natural fit for nested data (a list of lists, or nested JSON)?

Answers: 1. `return` hands a value to the caller; `print` only displays. 2. The default list is created once and persists across calls; use `items=None` then create inside. 3. `None`; and no — type hints are documentation (`mypy` checks them optionally). 4. A **base case** (stops) and a **recursive case** (calls itself on a smaller input, toward the base case). 5. `RecursionError` — Python has no tail-call optimization, so each pending call keeps a stack frame until the limit (~1000). 6. The data is *defined in terms of itself* (a list may contain lists), so a function defined in terms of itself mirrors its shape and can reach every level.

Session 4 — Exceptions, Files & Research Data

Part A — Exceptions

1. Which exception does `int("N/A")` raise? Wrap `int(value)` so it returns `None` on failure.
2. Why is a bare `except:` dangerous?
3. When should you use `raise` vs `assert`?

Part B — Files & Data

4. What does opening a file in `"w"` mode do to existing contents? Why prefer `with open(...)`?
5. After `csv.DictReader`, what type is each row?
6. CSV values read from a file are what type — and what must you do with numbers?

Answers: 1. `ValueError`; `try: return int(value) except (ValueError, TypeError): return None`.

2. It catches everything (even Ctrl+C and your own typos) and can hide bugs. 3. `raise` to validate real/untrusted input; `assert` for developer sanity checks (can be disabled). 4. Truncates it to empty immediately; `with` auto-closes even if the code crashes. 5. A `dict` keyed by the header row. 6. Strings; convert with `int()` / `float()`.

Session 5 — Regular Expressions, Modules & OOP

Part A — Regular Expressions

1. In regex, what does `.` match? How do you match a literal dot?
2. Why write regex patterns as raw strings `r"..."`?
3. What does `re.search` return when there's no match, and what must you do before `.group()`?

Part B — Modules & OOP

4. In a class, what is `self`?
5. What happens if you iterate a generator twice?
6. Why doesn't a module's `if __name__ == "__main__":` block run when you `import` it?

Answers: 1. Any character (except newline); use `\.` for a literal dot. 2. So backslashes aren't treated as Python string escapes. 3. It returns `None`; check `if m:` before `m.group()` or you'll hit an `AttributeError`. 4. The current instance ("this particular object"). 5. The second pass is empty — a generator is exhausted after one iteration. 6. On import, `__name__` is the module's name, not `"__main__"`, so the block is skipped.

Scoring guide (formative, not graded)

- **All correct, explained why:** ready for the capstone.
- **Right answer, fuzzy why:** re-do that session's `traps-and-gotchas` rows.
- **Wrong on Session 1 Part B items:** revisit the type traps before continuing — they're load-bearing.